

UNIDAD II. Uso de gramáticas.

2.1 Uso de la normalización de gramáticas para el diseño de compiladores

2.2 Análisis sintáctico descendente predictivo y gramáticas LL. (Left-to-Left)

2.3 Análisis sintáctico ascendente usando SLR, LALR o LR(1).

2.4 Otros métodos de transformación

UNIDAD II. Uso de gramáticas.

2.1 Uso de la normalización de gramáticas para el diseño de compiladores.

Una gramática formal define los elementos de un lenguaje y a cada lenguaje formal le corresponde una gramática.

$$G \Rightarrow L \Rightarrow G \text{ o } L(G)$$

UNIDAD II. Uso de gramáticas.

2.1 Uso de la normalización de gramáticas para el diseño de compiladores.

Una gramática libre de contexto (GLC) se define con los siguientes elementos:

- 1) Un conjunto de símbolos terminales.**
- 2) Un conjunto de símbolos no terminales.**
- 3) Un conjunto de producciones (donde cada producción tiene un símbolo no terminal en su parte izquierda y un conjunto de terminales o no terminales en su parte derecha).**
- 4) Un símbolo inicial o generador de la gramática (no terminal).**

UNIDAD II. Uso de gramáticas.

2.1 Uso de la normalización de gramáticas para el diseño de compiladores.

Ejemplo de la definición de la gramática formal de la instrucción mueve con dos parámetros A y B.

Mueve A, B $\rightarrow$ Esto significa que el contenido de la dirección A será asignado al contenido de la dirección B.

Para representar la gramática de esta única instrucción de: “Mueve” entre las variables “A” y “B” considerados como identificadores o direcciones tenemos la definición gramatical siguiente:

$\langle \text{Mueve} \rangle ::= \langle \text{Mueve} \rangle \langle \text{literal} \rangle, \langle \text{literal} \rangle \dots \dots \dots (1)$

$\langle \text{Mueve} \rangle ::= \langle \text{Mueve} \rangle | \langle \text{mueve} \rangle | \langle \text{MUEVE} \rangle \dots \dots (2)$

$\langle \text{literal} \rangle ::= \{A, B, C, \dots, Z \mid a, b, c, \dots, z\} \dots \dots (3)$

UNIDAD II. Uso de gramáticas.

2.1 Uso de la normalización de gramáticas para el diseño de compiladores.
En G se pueden observar las siguientes redundancias:

$S ::= Aa \mid B \mid D$

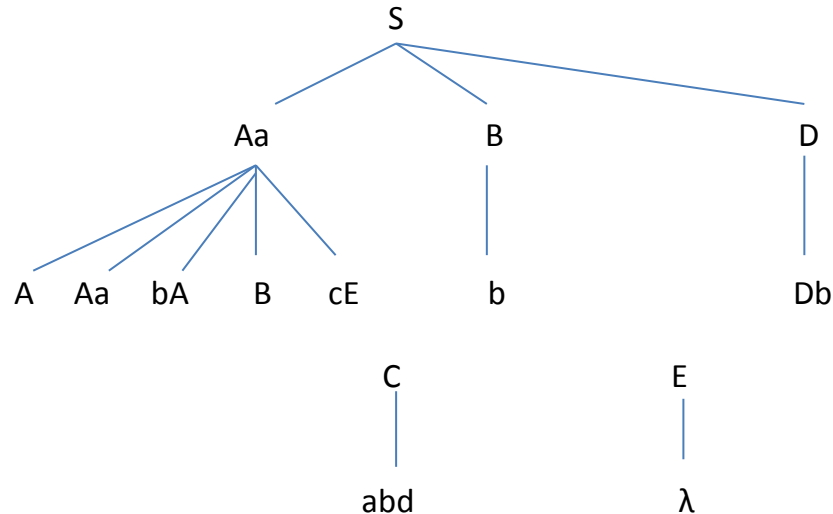
$B ::= b$

$A ::= A \mid Aa \mid bA \mid B \mid cE$ ($A ::= A$, $A ::= Aa$, $A ::= bA$, $A ::= B$ y $A ::= cE$)

$C ::= abd$

$E ::= \lambda$

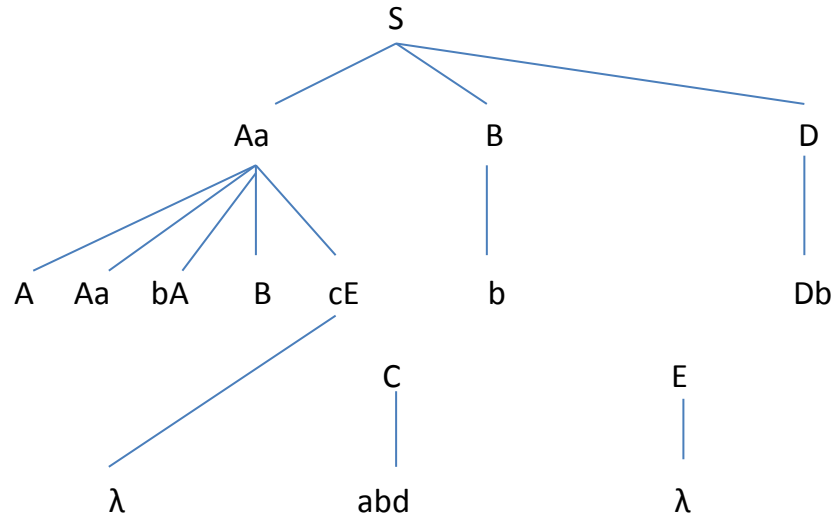
$D ::= Db$



UNIDAD II. Uso de gramáticas.

- 2.1 Uso de la normalización de gramáticas para el diseño de compiladores.
 En G se pueden observar las siguientes redundancias:

$S ::= Aa \mid B \mid D$ REGLA INNECESARIA
 $B ::= b$
 $A ::= A \mid Aa \mid bA \mid B \mid cE$ ($A ::= A$, $A ::= Aa$, $A ::= bA$, $A ::= B$ y $A ::= cE$)
 $C ::= abd$ $\longrightarrow$ SIMBOLO INNACCESIBLE (C)
 $E ::= \lambda$ $\longrightarrow$ REGLA NO GENERADORA
 $D ::= Db$ $\longrightarrow$ SIMBOLO SUPERFLUO (D)



UNIDAD II. Uso de gramáticas.

2.1 Uso de la normalización de gramáticas para el diseño de compiladores.

Reglas innecesarias

Una regla de la forma $A ::= A$ es innecesaria y puede ser eliminada

1. La regla $A ::= A$ es innecesaria.

Símbolos superfluos (o no generadores). Un símbolo superfluo es un símbolo no terminal A tal que no existe una derivación $A \rightarrow^* w$, donde $w \in \Sigma_T^*$.

Ejemplo D ($D ::= Db$)

2. Del símbolo D no se pueden derivar sentencias (símbolo superfluo).

UNIDAD II. Uso de gramáticas.

2.1 Uso de la normalización de gramáticas para el diseño de compiladores.

Símbolo inaccesible

3. Del símbolo C se puede derivar la sentencia abd, pero no es accesible desde S (símbolo inaccesible).

Regla no generadora

4. Las reglas $E ::= \lambda$ y $A ::= cE$ podrían ser reemplazadas por la regla $A ::= c$ (regla no generadora).

Regla de re-denominación

5. El símbolo B podría ser eliminado, y la regla $B ::= b$ podría ser reemplazada por las reglas $S ::= b$ y $A ::= b$ (regla de re-denominación).

UNIDAD II. Uso de gramáticas.

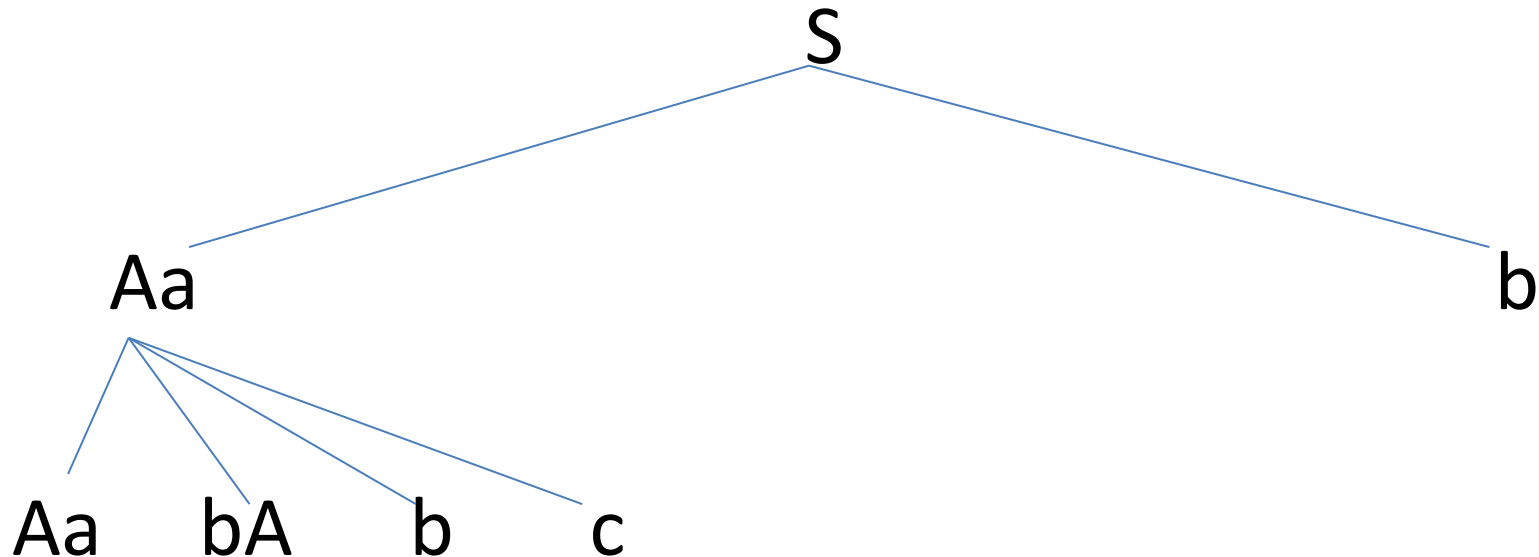
2.1 Uso de la normalización de gramáticas para el diseño de compiladores.

Si eliminamos estas redundancias en la gramática G , obtendremos una GIC G' tal que $L(G) = L(G')$:

$$S ::= Aa \mid b$$
$$A ::= Aa \mid bA \mid b \mid c$$

UNIDAD II. Uso de gramáticas.

2.1 Uso de la normalización de gramáticas para el diseño de compiladores.



UNIDAD II. Uso de gramáticas.

2.2 Análisis sintáctico descendente predictivo y gramáticas LL. (Left-to-Left)

Si al aplicar las reglas de producción se producen los reemplazos de símbolos no terminales que se encuentren más a la izquierda de la cadena en la que se debe producir el reemplazo.

Ejemplo: En el ejercicio anterior continuaremos aplicando reglas de producción y cuando se presente la alternativa de que símbolo aplicar, lo haremos eligiendo el que se encuentre más a la izquierda.

El conjunto de producciones es:

$\{(A \rightarrow 0B1), (A \rightarrow 1), (B \rightarrow 0A0), (B \rightarrow 01), (B \rightarrow 0AB)\}$

$A \rightarrow 0B1 \rightarrow 00AB1 \rightarrow 000A0B1 \rightarrow 0000B10B1 \rightarrow 0000110B1 \rightarrow 0000110011$
 $B \rightarrow 0AB \quad B \rightarrow 0A0 \quad A \rightarrow 0B1 \quad B \rightarrow 01$

>

UNIDAD II. Uso de gramáticas.

2.2 Análisis sintáctico descendente predictivo y gramáticas LL. (Left-to-Left) "PARSE"

Se define el parse como la secuencia ordenada de números (de reglas) aplicadas para construir dicho árbol.

Hay dos tipos de parse, que son:

El **parse-izquierdo**: Son los números de las reglas de derivación izquierda utilizadas para generar la cadena a partir del axioma, por tanto correspondiente a un análisis descendente.

El **parse-derecho**: Son los números de las reglas de derivación derecha utilizadas para generar la cadena a partir del axioma, en orden inverso. El tomar el orden inverso viene condicionado por ser el análisis ascendente el que normalmente utiliza las reglas de derivación derecha, con lo que el orden en el que aparecen al realizar el análisis es invertido.

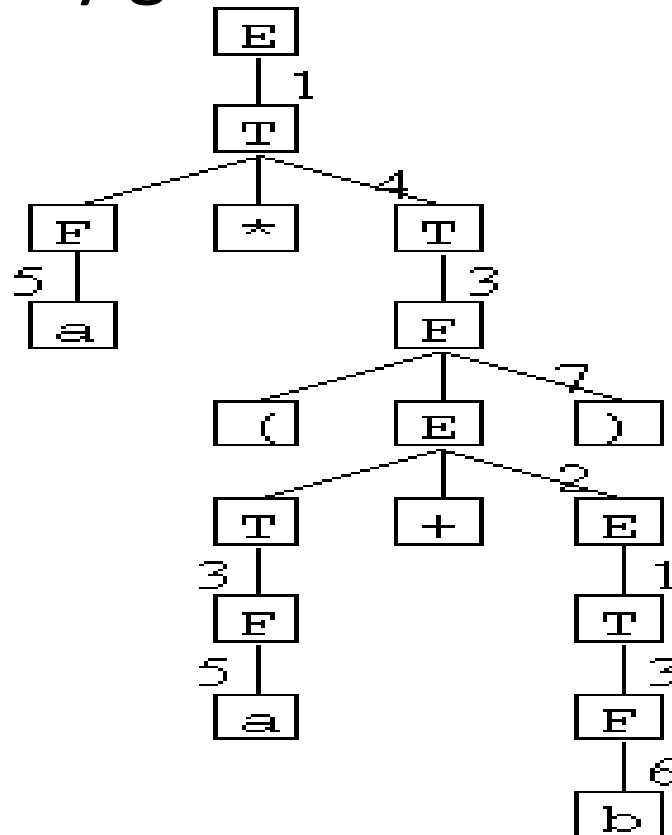
UNIDAD II. Uso de gramáticas.

2.2 Análisis sintáctico descendente predictivo y gramáticas LL. (Left-to-Left)

“PARSE”

Ejemplos

:



Dada la gramática

1. $E \rightarrow T$
2. $E \rightarrow T + E$
3. $T \rightarrow F$
4. $T \rightarrow F * T$
5. $F \rightarrow a$
6. $F \rightarrow b$
7. $F \rightarrow (E)$

y la sentencia " $a*(a+b)$ "

El parse izquierdo es:

1-4-5-3-7-2-3-5-1-3-6

y el derecho:

5-5-3-6-3-1-2-7-3-4-1

UNIDAD II. Uso de gramáticas.

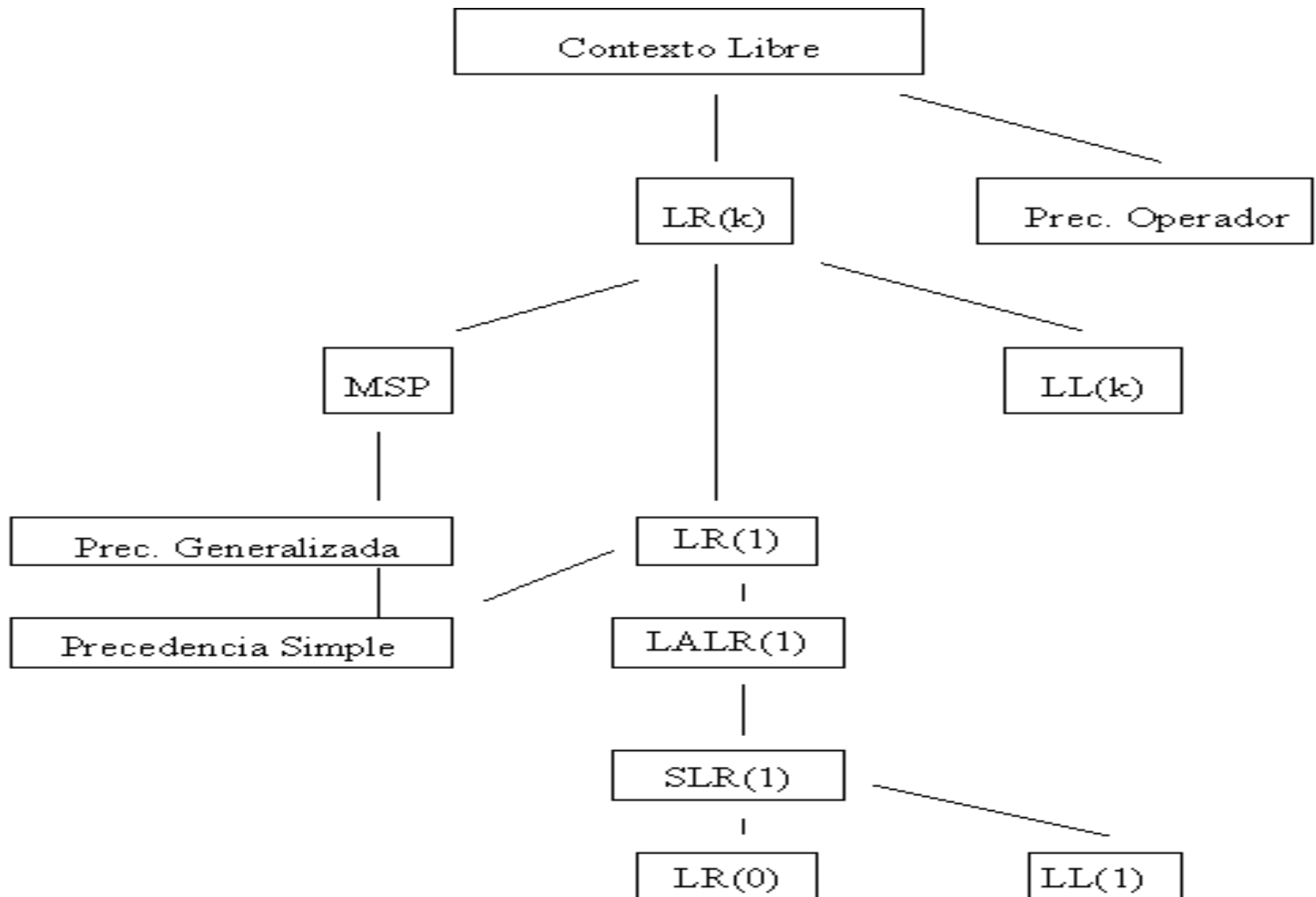
2.2 Análisis sintáctico descendente predictivo y gramáticas LL.(Left-to-Left)

Simultáneamente a la fase de análisis sintáctico, además de reconocer las secuencias de tokens, y analizar su estructura, pueden realizarse una serie de tareas adicionales, como:

- 1) Recopilar información de los distintos tokens y almacenarla en la tabla de símbolos.
- 2) Realizar algún tipo de análisis semántico, tal como la comprobación de tipos.
- 3) Generar código intermedio.
- 4) Avisar de los errores que se detecten.

UNIDAD II. Uso de gramáticas.

2.3 Análisis sintáctico ascendente usando SLR, LALR o LR(1).



Entrada y Salida de un Parser

Entrada: - (123.3 + 23.6)

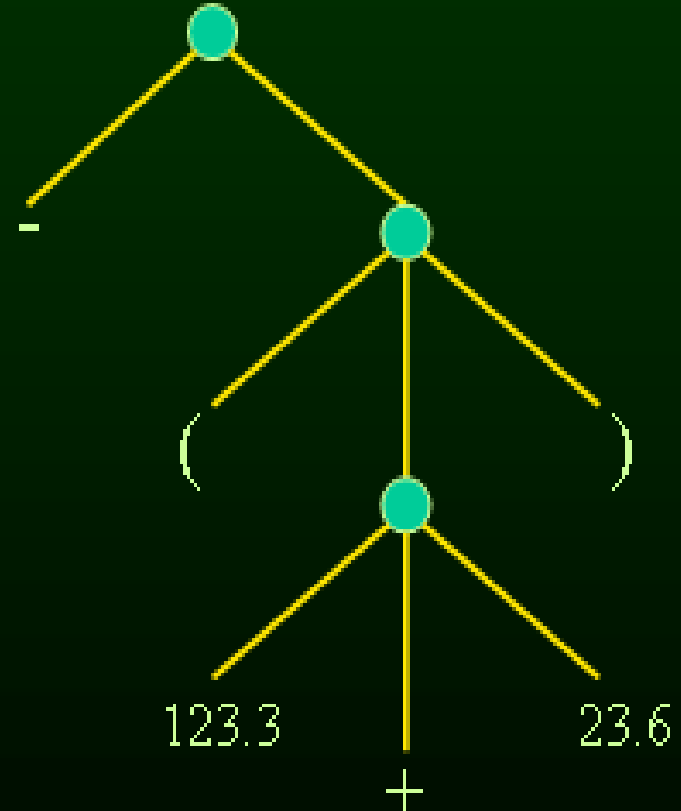
Token Stream

minus_op
left_paren_op
num(123.3)
plus_op
num(23.6)
right_paren_op



Analizador Sintáctico (Parser)

Arbol de Parseo



Leng. de Contexto Libre

```
graph TD; A[Leng. de Contexto Libre] --> B[Lenguajes Deterministas]; B --> C[Lenguajes LL(k)]; B --> D[Leng. de preced. simple]; C --> E[Lenguajes LL(1)]; D --> F[Leng. de preced. operador];
```

Lenguajes Deterministas

Lenguajes LL(k)

Leng. de preced. simple

Lenguajes LL(1)

Leng. de preced. operador

GLOSARIO

- GLC.-Gramáticas libres de Contexto
- MSP.- Mixed Strategy Precedence, techniques of McKeeman [12]. It was found that in addition to the main advantage of working for a larger class of grammars, our SLR(k) techniques are superior to the precedence techniques both in the speed of parser construction and in the size and speed of the parsers produced.
- SLR(k) (Simple Left to Right grammar without looking ahead more than k symbols) Gramática Libre de contexto con un recorrido de izquierda a derecha sin avanzar mas que un k símbolo, donde K es al menos un simbolo de precedencia.